day1-foundations

Day 1: Big Data Foundations and Distributed Systems

Answers to the 15 questions most likely to come up in a CAT or exam on Day 1 (big data concepts, distributed systems, CAP, and reference architectures), plus the Day 1 discussion question on MapReduce failures.

Q1. Define Big Data and explain the 5 V's.

Big Data is data whose size, variety or speed is too much for a single machine or a traditional database to store and analyse within acceptable time and cost. It is not just "a lot of data"; it is data that forces new tools and architectures.

V	Meaning	Example
Volume	Amount of data (terabytes to petabytes)	Years of bank transactions
Velocity	Speed at which data arrives and must be processed	Card payments checked for fraud in milliseconds
Variety	Structured, semi-structured and unstructured types	Tables, JSON logs, images, voice
Veracity	Quality and trustworthiness (noise, gaps, errors)	Duplicate or missing sensor readings
Value	Useful insight gained from the data	Lower stock-outs after demand forecasting

Volume and velocity stress the infrastructure, variety stresses integration, veracity stresses trust, and value is the reason to do the work at all.

Q2. Distinguish structured, semi-structured and unstructured data.

- **Structured:** fixed schema, rows and columns, easy to query with SQL (a relational sales table).
- **Semi-structured:** has tags or keys but no rigid table schema (JSON, XML, server logs).
- **Unstructured:** no predefined model (free text, images, audio, video).

Relational databases are built for structured data. Big Data platforms must handle all three, often in one pipeline, which is why NoSQL stores and schema-on-read tools exist.

Q3. Describe the four types of analytics with an example of each.

Type	Question answered	Example
Descriptive	What happened?	Monthly sales dashboard
Diagnostic	Why did it happen?	Drill-down shows sales fell in one region after a price change
Predictive	What is likely to happen?	Model forecasts next month's demand
Prescriptive	What should we do?	Optimiser recommends stock levels per store

Complexity and business value both rise from descriptive to prescriptive. Each level usually builds on the one below it.

Q4. Explain the Big Data analytics lifecycle.

It has six stages that repeat in a loop:

1. Business understanding: define the problem, goals and success measures.
2. Data acquisition: collect data from sources by batch or streaming ingestion.
3. Data preparation: clean, integrate and transform (often 60 to 80% of the effort).
4. Data analysis: explore, model and evaluate.
5. Visualisation and interpretation: present results to the right audience.
6. Action and feedback: apply the insight, measure the effect, feed lessons back to stage 1.

It is iterative because results from later stages change the questions asked in earlier ones.

Q5. What is a distributed system, and what are its key characteristics?

A distributed system is a group of independent computers that work together over a network and appear to users as one system.
Characteristics:

- Resource sharing (storage, CPU, data across nodes).
- Concurrency (many tasks run at the same time).
- Transparency (users do not see where data or work lives).
- Scalability (add nodes to add capacity).
- Fault tolerance (keeps working when some parts fail).
- Heterogeneity (nodes can differ in hardware and software).

Big Data uses distributed systems because one machine cannot hold or process the data fast enough, and clusters of cheap machines give scale, speed and reliability.

Q6. Compare vertical and horizontal scaling.

	Vertical (scale up)	**Horizontal (scale out)**
How	Bigger machine: more CPU, RAM, disk	More machines in a cluster
Limit	Hardware ceiling of one server	Practically very high
Cost at scale	Expensive	Cheap commodity hardware
Complexity	Low	Needs partitioning, coordination, consistency handling
Failure	Single point of failure	Survives node loss with replication

Big Data platforms (HDFS, Spark, Cassandra) are built for horizontal scaling.

Q7. State the CAP theorem and give examples of CP and AP systems.

The CAP theorem says a distributed data store cannot guarantee all three of these at the same time when a network partition happens:

- **Consistency:** every read sees the latest write.
- **Availability:** every request gets a response.
- **Partition tolerance:** the system keeps working when the network splits.

Partitions cannot be ruled out on a real network, so the practical choice during a partition is between C and A.

- CP (refuse or fail some requests rather than return stale data): HBase, MongoDB with majority write concern.
- AP (always answer, reconcile later): Cassandra, DynamoDB, Riak.
- CA only holds when there is no partition, such as a single-node relational database.

Example: a bank balance service prefers CP; a social media "likes" counter can be AP.

Q8. What does PACELC add to CAP?

CAP only describes behaviour during a partition. PACELC also covers normal operation:

- If there is a Partition (P), choose Availability (A) or Consistency (C).
- Else (E), choose Latency (L) or Consistency (C).

Even with a healthy network, strong consistency needs coordination between replicas, which adds delay. Cassandra is usually described as PA/EL (favours availability and low latency); HBase leans PC/EC.

Q9. Explain the main consistency models.

- Strong consistency: every read returns the most recent write. Simple to reason about, costs latency.
- Eventual consistency: replicas converge if no new writes arrive. Fast and available, reads may be stale for a short time.
- Causal consistency: operations that depend on each other are seen in order by everyone.
- Read-your-writes: a client always sees its own updates.
- Monotonic reads: a client never sees older data after seeing newer data.

Choice depends on the application: payments need strong consistency, a news feed can accept eventual consistency.

Q10. Compare ACID and BASE.

ACID (relational)	BASE (many NoSQL)
Atomicity: all or nothing	Basically Available: system keeps answering under failure
Consistency: valid state to valid state	Soft state: state can change without new input (replicas syncing)
Isolation: transactions do not interfere	Eventual consistency: replicas converge over time
Durability: committed data survives crashes	

ACID favours correctness, BASE favours availability and scale. Many modern systems mix the two (for example, ACID for orders, BASE for product catalogue).

Q11. Describe data partitioning strategies and their main challenge.

Partitioning splits data across nodes so work runs in parallel.

- Range partitioning: by value ranges (dates, ID ranges). Good for range queries, risk of hot spots on recent ranges.
- Hash partitioning: hash of a key decides the node. Even spread, but range queries must touch all nodes.
- List partitioning: explicit values map to partitions (region = East goes to node 2).
- Round-robin: rows dealt out in turn. Even load, no locality by key.

The main challenge is skew: when some keys hold far more data than others, one node becomes a bottleneck. Rebalancing as data grows is the second challenge.

Q12. Describe replication strategies.

Replication keeps copies of data on several nodes for fault tolerance, read throughput and lower latency.

- Master-slave (leader-follower): writes go to one master, reads can go to replicas. Simple, but the master is a bottleneck and a failover point.
- Multi-master: several nodes accept writes. Higher availability, but conflicts must be resolved.
- Quorum-based: a write succeeds after W replicas confirm, a read asks R replicas. If $W + R > N$ (number of replicas), a read overlaps the latest write. Example: $N = 3$, $W = 2$, $R = 2$.

Key design choices are replication factor, where copies are placed, and the consistency level.

Q13. Name the types of failure in a distributed system and the mechanisms used to tolerate them.

Failure types:

- Crash: a node stops.
- Omission: messages or operations are lost.
- Timing: replies arrive too late.
- Byzantine: a node behaves arbitrarily or maliciously.

Fault tolerance mechanisms:

- Replication: extra copies survive a lost node.
- Heartbeats: regular "I am alive" messages detect failed nodes.
- Checkpointing: saved state lets work restart from a known point.
- Retries and speculative execution: rerun failed or slow tasks elsewhere.

Q14. Compare batch, stream and interactive processing.

Model	Data	Goal	Examples
Batch	Bounded, stored dataset	High throughput, minutes to hours	MapReduce, Spark batch jobs
Stream	Unbounded, arriving continuously	Low latency, milliseconds to seconds	Kafka with Spark Structured Streaming
Interactive	Stored data, ad hoc questions	Answers in seconds	Hive, Spark SQL, Presto/Trino

Choose by whether the data is bounded, how fresh the answer must be, and the type of task. Many systems combine models.

Q15. Describe YARN, the layered reference architecture, and Lambda vs Kappa.

YARN manages cluster resources:

- ResourceManager: allocates resources for the whole cluster.
- NodeManager: one per node, launches and monitors containers.
- ApplicationMaster: one per application, requests containers and coordinates its tasks. It lets several frameworks (MapReduce, Spark, Tez) share one cluster.

Layered reference architecture: sources, ingestion, storage, processing, analytics and modelling, visualisation and consumption, with governance and operations (security, metadata, lineage, monitoring) running across all layers.

	Lambda	Kappa
Paths	Batch layer + speed layer + serving layer	One streaming path
Reprocessing	Rerun batch layer	Replay the stored stream (e.g. Kafka)
Strength	Accurate history plus fresh results	One code base, simpler
Weakness	Two code bases to keep in sync	Needs durable, replayable streams; costly for huge history

Q16 (discussion). How does MapReduce handle node failures?

- The master detects failed workers through missing heartbeats.
- Map tasks on a failed node are rerun on another node, even if they had finished, because their output lived on that node's local disk.
- Reduce tasks that had finished are not rerun, because their output is already in the distributed file system (GFS/HDFS).
- Input is replicated (three copies by default), so the rerun task reads another replica, ideally a local one.
- Slow tasks near the end of a job get backup copies (speculative execution); the first to finish wins.

These are Day 1 mechanisms in practice: heartbeats, replication, re-execution and speculation. Tasks must be deterministic so a rerun gives the same result.

Quick recall

- 5 V's: volume, velocity, variety, veracity, value.
- Analytics types in order: descriptive, diagnostic, predictive, prescriptive.
- Data preparation is usually 60 to 80% of project effort.

- Horizontal scaling (more machines) is the basis of Big Data platforms.
- CAP: during a partition, pick consistency or availability.
- CP examples: HBase, MongoDB (majority writes). AP examples: Cassandra, DynamoDB.
- PACELC: else (no partition), trade latency against consistency.
- ACID favours correctness; BASE favours availability and scale.
- Quorum rule: $W + R > N$ gives read-after-write overlap.
- Hash partitioning spreads load; range partitioning helps range queries; skew hurts both.
- YARN: ResourceManager, NodeManager, ApplicationMaster.
- Lambda has two processing paths; Kappa has one replayable stream.